

Received 16 January 2026, accepted 2 February 2026, date of publication 6 February 2026, date of current version 13 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3661859

RESEARCH ARTICLE

Denoising Colour Images Corrupted by Salt-and-Pepper Noise: Weighted Mean Filter With Iterative Approach and Deep Learning Modeling

MAHMOUD M. YAHAYA¹, POOM KUMAM¹, (Senior Member, IEEE),
AND KANOKWAN SITHITHAKERNGKIET²

¹Center of Excellence in Theoretical and Computational Science (TaCS-CoE), KMUTT Fixed Point, Research Laboratory, Room SCL 802 Fixed Point Laboratory, Science Laboratory Building, Department of Mathematics, Faculty of Science, King Mongkut's University of Technology Thonburi (KMUTT), Thung Khru, Bangkok 10140, Thailand

²Intelligent and Nonlinear Dynamic Innovations Research Center, Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok (KMUTNB), Bang Sue, Bangkok 10800, Thailand

Corresponding author: Poom Kumam (poom.kum@kmutt.ac.th)

This work was supported in part by the Petchra Pra Jom Klao Scholarship of King Mongkut's University of Technology Thonburi (KMUTT); in part by the Center of Excellence in Theoretical and Computational Science (TaCS-CoE), KMUTT; in part by KMUTT, Thailand Science Research and Innovation (TSRI), and National Science, Research and Innovation Fund (NSRF) Fiscal year 2024 under Grant FRB660073/0164; and in part by the NSRF via the Program Management Unit for Human Resources and Institutional Development, Research and Innovation under Grant B41G680025. The work of Mahmoud M. Yahaya was supported by the Petchra Pra Jom Klao Ph.D. Research Scholarship from KMUTT under Contract 24/2565. The work of Kanokwan Sithithakerngkiet was supported by the National Science, Research and Innovation Fund (NSRF), King Mongkut's University of Technology North Bangkok under Contract KMUTNB-FF-69-B-09.

ABSTRACT Image denoising has long been one of the most important and challenging problem in image processing, especially in the presence of high noisy levels. Considerable effort has been devoted to the development of effective denoising approaches. Over the past two decades, a notable strategy has emerged: a two-phase approach for grey-scale image denoising. In the first phase, an adaptive median filter (AMF)-like method is used to identify pixels that are likely corrupted by salt-and-pepper noise. In the second phase, a smooth objective function employed to restore the original pixel values while preserving image details and edges. In this paper, we first generalize the two-phase scheme to colour images. In the first phase, an adaptive weighted mean filter (AWMF) detects noisy pixels using an adaptively sized window. The window continuously expands until it reaches a predefined maximum or minimum window size; if a noisy pixel is identified, the process proceeds; otherwise, the pixel remains unaltered. In the second phase, we introduce and apply a smooth edge-preserving function for colour images. This function is solved via a proposed iterative algorithm that is simple to implement and globally convergent. We validated our approach through extensive experiments on diverse images with varying intensity levels and compared the proposed AWMF-MPPCG with existing impulse-noise denoising methods using six comparative metrics. The results demonstrate that our approach outperforms iterative-based methods in the literature, showcasing superior efficiency, effectiveness, and robustness in handling images with substantially high noisy levels. Secondly, we propose a data-driven, transformer-based neural network, referred to as FRESTORMER, for denoising images corrupted by salt-and-pepper noise, and we evaluate its performance against the AWMF-MPPCG and vector median filter method using both quantitative and qualitative metrics.

INDEX TERMS Impulse noise removal, dual-phase approach, iterative algorithm, line search, transformer-based architecture.

The associate editor coordinating the review of this manuscript and approving it for publication was Andrea F. Abate¹.

I. INTRODUCTION

In this article, we consider an image recovery problem, which is a task that has been, for decades, one of the most

challenging problems in image processing. There are many reasons that can cause images to become corrupted; some of these may include impulse noise, noisy sensors, error due to image transmission processes, and environmental factors. Noise adversely affects the quality of an image, leading to its degradation, loss of image features, and many more. These distortions are of grave concern in the domain of medical imaging such as Magnetic Resonance Imaging (MRI), Electrocardiogram (ECG) and Electroencephalography (EEG), where damage to the features of the diagnostic image occurs predominantly. There are mainly two categories of noises, namely: *additive Gaussian* noise and *impulse* noise. Impulse noise is the most prevalent form of noise; thus, in this work, we focus on recovering images impacted by impulse noise. This noise is mainly divided into “*salt and pepper*” and “*random-valued*” noises, respectively. In this form of noise, the noisy pixels can take maximum and minimum values in the case of *salt and pepper*, or they can be spread within a dynamic range, say for instance $[d_{min}, d_{max}]$ where, d_{min} and d_{max} represent the smallest and the largest pixel value respectively of an image, X .

Several methods have been developed for the removal of impulse noise; some well-known approaches include Standard Median Filtering (SMF) [19]. The SMF is a non-linear-based digital filter that performs denoising by replacing a central pixel value with a median of the pixels within a filtering window. Later, an extended variant of SMF, known as a vector median filter (VMF) [28], was introduced. The VMF approach exploits the inherent properties of pixels across channels, where each pixel is considered a c -dimensional vector-valued, c represents the number of channels (for instance, in colour images, $c = 3$). So, in VMF method, each image pixel within a predefined window size will be replaced by a processed median vector that is evaluated through some distance metric. However, the VMF method requires a computation of distance among all pixels, which may include both noisy and clean pixels. Consequently, this may create a blurring effect on the resulting denoised image. Other methods aimed at improving the VMF are discussed in [34], [33], and [32], and other references therein. In addition to static filter-based methods, other proposed methods achieved image restoration via learnable weights that encode generalizable image information obtained from a large-scale dataset for a specific task. The most popular approaches in this category include convolutional neural networks (CNNs) [41]-based architectures, Transformers [37]. These networks perform incredibly well when trained for a specific restoration task and under a given distortion scenario [42], [43], [44], [45]. Nonetheless, these approaches require retraining under slightly different distorted noise and are prone to forgetting [46].

Although the aforementioned approaches show a noticeable improvement in the image restoration task. They may not work well 50% in the case of filtering-based approaches or consistently generalize well when the noise ratio is very

high. One of the approaches that mitigates this shortcoming or reduces its impact, at least for filtering methods, is a variation of SMF known as an adaptive median filter (AMF), introduced in [20]. The AMF performs an additional layer of pinpointing the noisy pixel values and then adopts an increasing sliding window size approach to identify a noisy pixel candidate that would be replaced by a noiseless median pixel. The window size increases multiple times until a suitable clean pixel is identified or a maximum window size is achieved, in which case the process is repeated. This method performs better than SMF. Nevertheless, the recovery effect may be poor when the noise is too high. The method may take considerable time due to the increasing window size requirement of the scheme. In AMF, each corrupted pixel candidate will be replaced with another potentially noiseless pixel that is slightly from a faraway location. Thus, there is a high potential for the edge information and smoothness of the image to be lost. Consequently, the resulting restored image may appear blurry. Some ideas that aim to overcome some of these drawbacks include symmetric and asymmetric trimming of filters [21], [22], [22], [23], e.t.c. Nonetheless, despite the improvements achieved by earlier methods, the finer edge details of the restored image may not be preserved. Thus, the recovered image may ultimately be distorted. Alternative methods that mitigate edge information loss in a restored image are known as variational methods. These approaches have been shown to potentially preserve fine edges without significantly distorting the restored image that is corrupted by outliers and impulse noise [8]. However, for variational methods to work effectively, they require shifting or changing all pixels in the corrupted image, which may also affect clean pixels.

In an effort to overcome the aforementioned challenges while simultaneously leveraging the advantages of AMF-based methods and the benefits of variational methods, Raymond et al. [9] introduced a combined two-phase algorithm that serves two purposes. In the first stage, an adaptive median filter is used to identify all the likely corrupted pixel values. In the second stage, the noisy image is recovered by solving a newly formulated regularized objective functional, denoted by $\mathcal{F}$, which also includes an l_1 -data fitting term. The regularization component of $\mathcal{F}$ is made up of an edge-preserving function $\psi_\delta : \mathbb{R}^+ \rightarrow \mathbb{R}^+$, where $\delta > 0$. The second stage aims to apply the objective function $\mathcal{F}$ solely to the pre-identified corrupted pixels. Thus, the final restored image will have a preserved edges and features, while the clean pixels maintain their original texture or remain unchanged.

The approach developed in [6] demonstrated remarkable performance in recovering a noisy images with a higher proportion, even in cases of severe *salt and pepper* impulse noise. Moreover, the method was experimentally shown to outperform both AMF-based approaches and variational-based approaches independently, as the two-phase method accurately preserves the finer details, edges, and

texture of the image even when the noise ratio reaches 90%. We describe the two-phase approach of Chan et al. [6] intuitively in Fig. 1.

It is worth noting that there are some other improvements to AMF that have been proposed to mitigate potential challenges that may arise. As noted by Zhang and Fang [7], these drawbacks include the likelihood of error in the noise detection stage. An error in noise detection may occur when the identified noisy pixel value at the centre of a given window is either the local maximum or the local minimum within that window. However, there is no salt or pepper noise, yet the pixel value is still designated as a noisy candidate. Secondly, the AMF may lose certain essential image details and edge information in images with a higher noise ratio. An adaptive weighted mean filter (AWMF) was introduced to overcome the shortcomings of the AMF approach. Instead of replacing the noisy candidates with a median value, as in the AMF-base method, the AWMF reduces the detection errors and replaces the noisy candidates with a weighted mean value.

On the other hand, the functional objective, $\mathcal{F}$, is made up of a non-smooth data fitting term which is costly to solve; consequently Chan et al. [9] proposed a smooth version of the objective function by simply removing the non-smooth component, without affecting the quality of the restored images.

The smooth version of the regularized objective function can be solved using various smooth-based descent optimization algorithms as reported, for example, by Jinkui et al. [14]. The authors introduced a spectral gradient-based method for impulse noise recovery. Other descent-based method that has been successfully used is conjugate gradient (CG) method. The CG is a first-order method widely used for unconstrained optimization problems of varying difficulty. The algorithm was initially developed for convex quadratic problems, in which the CG method constructs search directions that are orthogonal to all previously generated directions through a Gram-Schmidt orthogonalization procedure applied to the gradients. The CG was later extended to general non-linear optimization problems by Fletcher-Reeves [24]. A number of CG methods have been proposed for image recovery tasks [2], [11]. In particular, Cheng [25] introduced a modification of PRP-CG method in which the generated search direction satisfy the so-called sufficient descent property without imposing restrictive assumptions. Furthermore, Yu et al. [2] studied hybrid CG projection methods as well as spectral CG approaches for impulse noise recovery problems, and showed that their search directions satisfy a sufficient descent under some additional assumptions. Recently introduced methods include [13], [14], [15], [16].

However, the methods discussed above are limited to a two-phase impulse noise recovery paradigm that depends on the AMF method for identifying corrupted pixels in greyscale images, thus constraining their generality and practical application. To our knowledge, the improved AMF

variant, referred to as AWMF, has not yet been explored, and the formulation of the edge-preserving objective function remains limited to greyscale images. Accordingly, it is worthwhile to formulate a colour-based functional objective and enhance AWMF for application to colour images in the noise detection stage, followed by the development of an efficient algorithm to solve the formulated functional. Thus, we proposed a two-stage impulse noise recovery model for colour images. We developed a fast and efficient CG algorithm that is simple to implement and requires fewer computations for colour denoising problem (with impulse noise).

We concisely summarize the contributions presented in this manuscript.

- We introduce a new dual-phase colour image denoising approach departs from existing strategies: beginning with [6] and [9] and including recent improvements [14], [15], [16]-which were developed for grey-scale images and rely on Adaptive Median Filters (AMF). Our approach named: AWMF-MPPCG is formulated using an Adaptive Weighted Mean Filter in the noise detection phase. Additionally, in the noise removal phase, we developed an efficient CG-based algorithm, MPPCG that solves an extended, differentiable objective functional specifically formulated for colour images.
- The introduced MPPCG algorithm is efficient and requires fewer computational operations. Thus, it is fast, as indicated by the preliminary experiments.
- Moreover, we also consider finetuning a transformer-based model on salt-and-pepper noisy images. The denoising variant of the model named RESTORMER [36] was originally trained specifically for Gaussian noise with a small noise level. Thus, we finetuned it on images corrupted with small to high noise levels of salt-and-pepper and compare it with other methods introduced for impulse noise removal.
- Considerable effort has been given to experimentation, where a wide variation of images taken from multiple sources is used to showcase a comparative performance of FRESTORMER and AWMF-MPPCG with other recently introduced methods for salt-and-pepper noise.

The article is compartmentalized into sections, which include a preliminary and model formulation Section II. An algorithmic scheme, which gives a step-by-step breakdown of the proposed improved CG-based algorithm and its convergence properties, is presented in Section III. Motivated by [36], we visualized and discussed the finetune RESTORMER architecture in Section IV. In Section V, we describe the experimentation to empirically illustrate the performance of the proposed approaches. Finally, we give some concluding remarks in Section VI. Unless otherwise, we defined $\|\cdot\|$ to denote an Euclidean norm.

II. MODEL FORMULATION

Over the years, there has been a considerable effort in developing various improved methods for restoring images with a degree of noise. These proposed methods are mostly adaptive in their construction. One of the prominent strategies is known as *Adaptive median filter* [20] applied to a grey-scale image is described as follows:

Suppose we denote the uncorrupted image as $\mathbf{x}$ which possesses a $M \times N \times C$ pixels values, where C is the number of channels, (i.e., $C = 3$), also let $\mathcal{I} := \{1, 2, 3, \dots, M\} \times \{1, 2, 3, \dots, N\} \times \{1, 2, 3\}$ be the indexing set of the original image, $\mathbf{x}_{i,j,k}$ to represent an observed pixel value located at position $(i, j, k) \in \mathcal{I}$. Suppose all the pixel values, $x_{i,j,k}$ are bounded above and below within a dynamic range, $[d_{min}, d_{max}]$. Thus, we defined a pixel value, $y_{i,j,k}$ of a noisy image model, $\mathbf{Y}$ corrupted with salt and pepper noise as follows:

$$y_{i,j,k} := \begin{cases} d_{max}, & \text{with probability } p_s \\ d_{min}, & \text{with probability } p_p \\ x_{i,j,k}, & \text{with probability } 1 - l, \end{cases} \quad (1)$$

where p_s is a probability of salt noise, p_p is probability of pepper noise and $l := p_s + p_p$ represents the noise level. Note that p_s in AMF, a window, say $S_{(i,j)}(z)$ of size, say, $z \times z$, is dynamically constructed with a centered position (i, j) . Now suppose $S_{(i,j)}^{med}(z)$ is a median value of the window, $S_{(i,j)}(z)$, then according to a threshold, the AMF algorithm replaces a pixel value with its median value, $S_{(i,j)}^{med}(z)$ if it is corrupted, otherwise it remain unchanged.

A. ADAPTIVE WEIGHTED MEAN FILTER

Although the AMF performs considerably well in removing noise from corrupted images, it may have some drawbacks, as pointed out in [7]. These shortcomings include the possibility of mishaps in detecting a noisy candidate pixel, where the authors gave an example that led to this error. Additionally, AMF may miss some important details and edge information in the image [7]. These challenges motivated the development of the adaptive weighted mean filter, AWMF [7].

Suppose for a particular channel k , we denote the $S_{(i,j,k)}^{max}(z)$ and $S_{(i,j,k)}^{min}(z)$ as the *maximum* and *minimum* of a given window, $S_{(i,j,k)}(z)$ respectively. In AWMF, a weight, $w_{q,l,k}$ value defined by

$$w_{q,l,k} = \begin{cases} 1, & S_{(i,j,k)}^{min}(z) < y_{q,l,k} < S_{(i,j,k)}^{max}(z) \\ 0 & \text{otherwise,} \end{cases} \quad (2)$$

is used in formulating the weighted mean value, $S_{(i,j,k)}^{mean}(z)$, define as follows:

$$S_{(i,j,k)}^{mean}(z) = \begin{cases} \frac{\sum_{(q,l,k) \in S_{(i,j,k)}(z)} w_{q,l,k} * y_{q,l,k}}{\sum_{(q,l,k) \in S_{(i,j,k)}(z)} w_{q,l,k}}, & \sum_{(q,l,k) \in S_{(i,j,k)}(z)} w_{q,l,k} \neq 0, \\ -1, & \text{otherwise.} \end{cases} \quad (3)$$

Thus, the filter performs *noise detection* by initially forming an increasing adaptive window size, which enlarges until a maximum window size, $z_{max} \times z_{max}$, is reached. A pixel value located at a centre of the window is noisy if it is either the $S_{(i,j,k)}^{min}(z)$ or $S_{(i,j,k)}^{max}(z)$, else the pixel is noiseless. Immediately after the corrupted pixel detection stage, the identified altered pixel will be replaced by the introduced weighted mean value, while the unaltered pixel will remain untouched.

In light of [7], the AWMF algorithm is outline as follows:

Algorithm 1 Adaptive Weighted Mean Filter, AWMF

Inputs: Take the corrupted image, $\mathbf{Y}$ with pixel value, $y_{i,j,k}$

for each $(i, j, k) \in \mathcal{I}$ as input, initialize window size, $z = 3$, maximum window size, $z_{\#}$

Output: Recovered pixels $\bar{\mathbf{x}}$ of the image, with pixel value $\bar{x}_{i,j,k}$ for $(i, j, k) \in \mathcal{I}$.

Step 1: Evaluate $S_{(i,j,k)}^{min}(z)$, $S_{(i,j,k)}^{max}(z)$, $S_{(i,j,k)}^{min}(z+1)$, $S_{(i,j,k)}^{max}(z+1)$ and then $S_{(i,j,k)}^{mean}(z)$ using (3).

Step 2: If $S_{(i,j,k)}^{min}(z) = S_{(i,j,k)}^{min}(z+1)$, $S_{(i,j,k)}^{max}(z) = S_{(i,j,k)}^{max}(z+1)$, and $S_{(i,j,k)}^{mean}(z) \neq -1$, go to **Step 4**, else, $z = z + 1$.

Step 3: If $z \leq z_{max}$, go to **Step 1**; else, recovered pixel

value, $\bar{x}_{i,j,k} := S_{(i,j,k)}^{mean}(z)$, and stop.

Step 4: If $S_{(i,j,k)}^{min}(z) < y_{i,j,k} < S_{(i,j,k)}^{max}(z)$ then $y_{i,j,k}$ is noiseless, and set $\bar{x}_{i,j,k} := y_{i,j,k}$; else, $y_{i,j,k}$ is noisy, set $\bar{x}_{i,j,k} := S_{(i,j,k)}^{mean}(z)$, and stop.

Experimental results obtained using Algorithm 1 indicate a remarkable improvement in performance over AMF. We added a Python implementation of AWMF and AMF, which are used in the noisy pixel detection phase, together with a variational approach that will discussed in the next Subsection. This implementation is currently accessible via a link.¹ Thus, we improve this approach by adding a variational-like layer for colour image denoising. Next, we describe the objective formulation for grey-scale images and then extend the formulation to colour imaging.

B. VARIATIONAL APPROACH

Variational methods [6], [8] perform image recovery from a corrupted image through minimizing an objective functional $\mathcal{F}$, which consists of l_1 data fitting term and edge-preserving regularization component, ψ_δ . Let $\mathcal{N}_{e,i,j} := \{(i, j-1), (i, j+1), (i-1, j), (i+1, j)\}$ be a set that contains all the four neighbours of a pixel at index (i, j) , the corrupted pixels set denoted by, $\mathcal{N}_{nc} := \{(i, j) \in \mathcal{I} : y_{i,j} \neq \bar{y}_{i,j}, y_{i,j} \in [d_{min}, d_{max}]\}$, while the set of noiseless pixels is simply

¹<https://github.com/MAHMOUDPD/image-denoising>

$\mathcal{N}'_{nc} := \mathcal{I} \setminus \mathcal{N}_{nc}$ then, $\mathcal{F}$ is defined as follows:

$$\mathcal{F}_{\psi_\delta}(\tilde{\mathbf{x}}) := \sum_{(i,j) \in \mathcal{N}_{nc}} \left\{ |x_{i,j} - y_{i,j}| + \frac{\lambda}{2} (S^1(\tilde{\mathbf{x}}) + S^2(\tilde{\mathbf{x}})) \right\}, \quad (4)$$

where $\lambda > 0$ is the positive parameter that is chosen based on [8],

$$\begin{aligned} S^1(\tilde{\mathbf{x}}) &= \sum_{(m,n) \in \mathcal{N}_{e_{i,j}} \setminus \mathcal{N}_{nc}} 2\psi_\delta(\tilde{x}_{i,j} - y_{m,n}), \\ S^2(\tilde{\mathbf{x}}) &= \sum_{(m,n) \in \mathcal{N}_{e_{i,j}} \cap \mathcal{N}_{nc}} \psi_\delta(\tilde{x}_{i,j} - \tilde{x}_{m,n}), \end{aligned} \quad (5)$$

in which ψ_δ is edge preserving function. The objective function, defined in (4), is non-smooth due to the l_1 data fitting term, which may be difficult to optimize. Thus, in [9], the authors postulated that the term $|x_{i,j} - y_{i,j}|$ can be dropped without affecting the denoised image, since it was shown to enable detection of noisy pixels, which in our approach is handled in the first phase. Consequently, the smooth formulation of (4) is redefined as follows

$$\tilde{\mathcal{F}}_{\psi_\delta}(\tilde{\mathbf{x}}) := \sum_{(i,j) \in \mathcal{N}_{nc}} \left\{ S^1(\tilde{\mathbf{x}}) + S^2(\tilde{\mathbf{x}}) \right\}, \quad (6)$$

where the terms in (6) $S^1(\tilde{\mathbf{x}})$ and $S^2(\tilde{\mathbf{x}})$ are defined in the same way as in (5). Now, motivated by the formulation crafted (6) for the grayscale image, we came up with the following extended re-formulation for the functional objective for colour image given by:

$$\mathcal{G}_{\psi_\delta}(\tilde{\mathbf{x}}) := \sum_{k=1}^c \sum_{(i,j) \in \mathcal{N}_{nc}} \left\{ S^1(\tilde{\mathbf{x}}) + S^2(\tilde{\mathbf{x}}) \right\}, \quad (7)$$

where the outer sum runs over the three channels, c , of the colour image, $\tilde{\mathbf{x}}$ obtained from the filtering stage via AWMF and we define the edge-preserving function, $\psi_\delta : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ by a Huber loss function [10] given by

$$\psi_\delta(\tilde{x}) = \begin{cases} \frac{\tilde{x}^2}{2} & \text{for } |\tilde{x}| \leq \delta \\ \delta(|\tilde{x}| - \frac{1}{2}\delta) & \text{for } |\tilde{x}| > \delta, \end{cases}$$

and

$$\psi'_\delta(\tilde{x}) = \begin{cases} \tilde{x} & \text{for } |\tilde{x}| \leq \delta \\ \delta \text{sign}(\tilde{x}) & \text{for } |\tilde{x}| > \delta, \end{cases} \quad (8)$$

where $\tilde{x}$ denotes a colour pixel value, and the $\delta > 0$ is chosen such that $|\tilde{x}|$ is sufficiently approximated by $\psi_\delta(\tilde{x})$. Since the $\psi_\delta(\tilde{x})$ in (8) is convex, coercive and its derivative, $\psi'_\delta(\tilde{x})$ is Lipschitz continuous then as proven in Propositions 5-6 in [11] the functional defined in (6) has Lipschitz continuous gradients and similarly the gradient of the extended formulation, $\mathcal{G}_{\psi_\delta}$ in (7) for coloured images, is also Lipschitz continuous. As formalized in [11], we have

Proposition 2.1: Suppose the edge-preserving function $\psi_\delta(\tilde{x})$ is even, has Lipschitz continuous gradients, and strictly increasing with respect to $|\tilde{x}|$, then functional $\mathcal{G}_{\psi_\delta}$

possesses a global minimum, whose minimizer, $\tilde{\mathbf{x}}^* \in [d_{min}, d_{max}]$, $\forall (i,j) \in \mathcal{N}_{nc}$.

The minimization of the new functional, $\mathcal{G}_{\psi_\delta}$ can be achieved using first-order optimization methods; in particular, in this article, we consider formulating a conjugate gradient method (CGM) which is suitable for large-scale unconstrained optimization, easy to implement and globally convergent.

III. PROPOSED ALGORITHM

In this section, we discuss the *two stage* approach, which is motivated from framework proposed by Chan et al. [6]. Thus, our introduced approach comprises two steps: *corrupted pixel detection* and *noisy pixel replacement*.

- We use *Adaptive Weighted Mean Filter*, AWMF on the noisy image $\mathbf{Y}$ corrupted by salt and pepper to obtain an image $\tilde{\mathbf{Y}}$ with entries $\tilde{y}_{i,j}$ in the *noisy detection* step.
- In the *replacement* step, the noiseless pixels which are elements of the complement, $\mathcal{N}'_{nc}$ will be unaltered, on the other hand, suppose for an entry with indices (i,j) of $\mathcal{N}_{nc}$, then one of its neighbours say $(m,n) \in \mathcal{N}_{e_{i,j}}$ can either be noiseless pixel value, hence it will be unaltered otherwise, it is a noisy pixel value, in which case it would be restored. The recovery of the corrupted pixel values will be done by minimizing the functional, $\mathcal{G}_{\psi_\delta}(\tilde{\mathbf{x}})$ defined in (7) where the neighbourhood set, $\mathcal{N}_{e_{i,j}}$ is partitioned into two sets: i.e., $\mathcal{N}_{e_{i,j}} \cap \mathcal{N}_{nc}$ and $\mathcal{N}_{e_{i,j}} \cap \mathcal{N}'_{nc}$. Therefore, we set $\tilde{x}_{m,n} = y_{m,n}$ provided that the indices $(m,n) \in \mathcal{N}_{e_{i,j}} \cap \mathcal{N}'_{nc}$, however, if $(m,n) \in \mathcal{N}_{e_{i,j}} \cap \mathcal{N}_{nc}$ then recovery of $y_{m,n}$ must be carried out.

Next, we craft out a CGM for minimizing $\mathcal{G}_{\psi_\delta}(\tilde{\mathbf{x}})$. Suppose we have $\mathcal{G}_\psi : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex, continuous and differentiable, then we have a model for unconstrained optimization stated as

$$\min_{\tilde{\mathbf{x}} \in \mathbb{R}^n} \mathcal{G}_\psi(\tilde{\mathbf{x}}), \quad (9)$$

where we represent the gradient of (9) at $\tilde{\mathbf{x}}$ by g i.e., $g(\tilde{\mathbf{x}}) := \nabla \mathcal{G}_\psi(\tilde{\mathbf{x}})$. As was hinted in the introduction, the CG method is among the most popular methods for solving (9) due to its minimum storage requirement and simple iterative formulae given below:

$$\tilde{\mathbf{x}}^{(k+1)} = \tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)}, \quad k = 0, 1, \dots \quad (10)$$

where $\alpha^k > 0$ is a step length chosen using some line search techniques, and the search direction of the CG methods is generally defined as

$$d^{(k)} = \begin{cases} -g^{(k)} & k = 0, \\ -g^{(k)} + \beta^{(k)} d^{(k-1)} & k \geq 1, \end{cases} \quad (11)$$

where $g^{(k)} = \nabla \mathcal{G}_\psi(\tilde{\mathbf{x}}^{(k)})$ and the term $\beta^{(k)}$ is known as the CG parameter whose choice is significant in determining the performance of CG method and varies for each CG method.

Algorithm 2 A New Conjugate Gradient (MPPCG) Algorithm

Inputs: $\tilde{\mathbf{x}}^{(0)}$, $\sigma \in (0, \frac{1}{2})$, $\rho \in (0, 1)$,
 $0 \leq \rho < \frac{1}{\sigma}$, $\epsilon > 0$, $\alpha^{(0)} = 1$, and set $k := 0$.

Output: $\tilde{\mathbf{x}}^*$

Step 0: Compute $g^{(0)}$, and set $d^{(0)} \leftarrow -g^{(0)}$.

Step 1: If $\|g^{(k)}\| \leq \epsilon$ do

Choose, α^k that satisfy the following conditions:

$$\mathcal{G}_\psi(\tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)}) \leq \mathcal{G}_\psi^{(k)} + \rho \alpha^k g^{(k)T} d^{(k)}, \quad (13)$$

$$g(\tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)})^T d^{(k)} \geq \sigma g^{(k)T} d^{(k)}, \quad (14)$$

Compute,

$$d^{(k)} := -g^{(k)} + \beta^{(k)} d^{(k-1)}, \quad (15)$$

where $\beta^{(k)}$ is defined in (12)

Step 2: Update $\tilde{\mathbf{x}}^{(k+1)} \leftarrow \tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)}$.

Step 3: Set $k \leftarrow k + 1$ and go to **Step 1**

In this article, we defined $\beta^{(k)}$ ² as

$$\beta^{(k)} := \frac{\rho \|g^{(k)}\|^2}{(\rho d^{(k-1)T} g^k - g^{(k-1)T} d^{(k-1)} + 1/\alpha^k) \wedge \|g^k\| \|d^{(k-1)}\|} \quad (12)$$

All is now set to present the proposed conjugate descent variant named MPPCG Algorithm. After initializing terms, the MPPCG computes the steepest descent direction represented by $d^{(0)}$. The algorithm stops when the condition $\|g^{(k)}\| \leq \epsilon$ is satisfied; otherwise, it continues by first evaluating a suitable step-size α^k that satisfies (13) and (14). Subsequently, the algorithm computes the conjugate gradient direction (15). In Step 1, the algorithm uses the evaluated components α^k and $d^{(k)}$ to update $\tilde{\mathbf{x}}^{(k+1)}$ in Step 2 and increments the iteration counter in Step 3 and then return to Step 1.

Remark 3.1: The input $\tilde{\mathbf{x}}^{(0)}$ to the iterative procedure 2 is a vectorized noisy pixels which are identified concurrently during the initial filtering stage obtained from the improved AWMF algorithm 1. This vector of noisy candidates passes through iterative correction. The entries of the final approximate solution achieved through the iterative algorithm replace the noisy pixel values of the salt-and-pepper noisy image. The obtained step-size, α^k in (13) is evaluated using the following outlined algorithmic pseudo-code

Assumption 3.2: Let $\tilde{\mathbf{x}}^{(0)} \in \mathbb{R}^n$ represent an initial starting point. The objective function, $\mathcal{G}_\psi$ at $\tilde{\mathbf{x}} \in \mathbb{R}^n$ is bounded below on the level set $\mathcal{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^n | \mathcal{G}_\psi(\tilde{\mathbf{x}}) \leq \mathcal{G}_\psi(\tilde{\mathbf{x}}^{(0)})\}$. Moreover, throughout some neighbourhoods of $\mathcal{S}$, the objective function $\mathcal{G}_\psi$ is smooth and its gradient is Lipschitz continuous.

Remark 3.3: We can deduce the following remarks from Assumption 3.2 made above.

²The symbols $\vee$ and $\wedge$ usage: let $a, b \in \mathbb{R}$ then $a \vee b := \max\{a, b\}$ while $a \wedge b := \min\{a, b\}$

Algorithm 3 Weak Wolfe Line Search [26]

Inputs: Initial step-size $\alpha^0 > 0$, parameters ρ and σ that satisfy

$$0 \leq \rho < \frac{1}{\sigma}$$

$$\alpha_{\text{low}} \leftarrow 0$$

$$\alpha_k \leftarrow \alpha^0$$

$$\alpha_{\text{upp}} \leftarrow +\infty$$

while True

if $\mathcal{G}_\psi(\tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)}) > \mathcal{G}_\psi^{(k)} + \rho \alpha^k g^{(k)T} d^{(k)}$

$\alpha_{\text{upp}} \leftarrow \alpha^k$;

elseif $g(\tilde{\mathbf{x}}^{(k)} + \alpha^k d^{(k)})^T d^{(k)} < \sigma g^{(k)T} d^{(k)}$

$\alpha_{\text{low}} \leftarrow \alpha^k$

else

return α^k

end if

if $\alpha_{\text{low}} \leftarrow +\infty$

$\alpha^k \leftarrow 2\alpha^k$

else

$\alpha^k \leftarrow \frac{1}{2}(\alpha_{\text{low}} + \alpha_{\text{upp}})$

end if

end while

return α^k

- (i) Suppose $\tilde{\mathbf{x}}^{(k)}$ and $\tilde{\mathbf{x}}^{(k-1)}$ are any two varying iterates in the neighbourhood $\hat{\mathcal{S}}$ of $\mathcal{S}$. Then respective gradients at those points of $\mathcal{G}_\psi$ satisfies

$$\|g^{(k)} - g^{(k-1)}\| \leq L \|\tilde{\mathbf{x}}^{(k)} - \tilde{\mathbf{x}}^{(k-1)}\|, \quad L > 0. \quad (16)$$

- (ii) Moreover, we have boundedness of the sequences of gradient $\{g^{(k)}\}$ and iterates $\{\tilde{\mathbf{x}}^{(k)}\}$ respectively. That is, there exists some constants $h_1 > 0$, $h_2 > 0$ such that

$$\|g^{(k)}\| \leq h_1, \quad \text{and} \quad \|\tilde{\mathbf{x}}^{(k)}\| \leq h_2, \quad \forall k. \quad (17)$$

Now, we start a Lemma, which shows that the proposed search direction (15) is sufficiently descent.

Lemma 3.4: Suppose we have some positive constants $\sigma \in (0, 1)$ and $\rho \in (0, 1)$. Then, the sequence $\{d^{(k)}\}$ as defined in (15) generated by Algorithm 2 is sufficiently descent, i.e., there exists a positive constant set as $m = (1 - \rho)$ such that

$$g^{(k)T} d^{(k)} \leq -m \|g^{(k)}\|^2, \quad \forall k, \quad (18)$$

is satisfied.

Next, we prove the global convergence result of the proposed MPPCG algorithm with the aid of Lemmas.

Lemma 3.5: Let Assumption 3.2 holds. Then the sequence $\{d^{(k)}\}$, of search directions generated by Algorithm 2 is bounded.

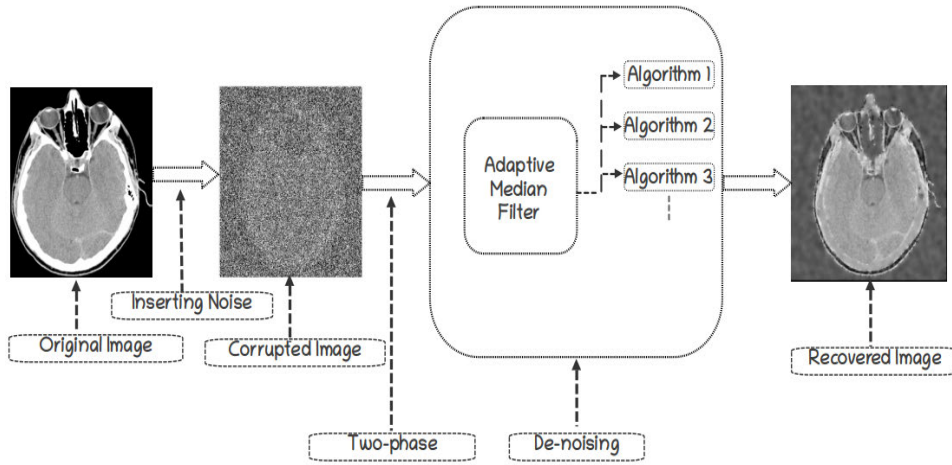


FIGURE 1. The figure shows an intuitive structure of a two-stage/two-phase method for recovery of a corrupted grey image.

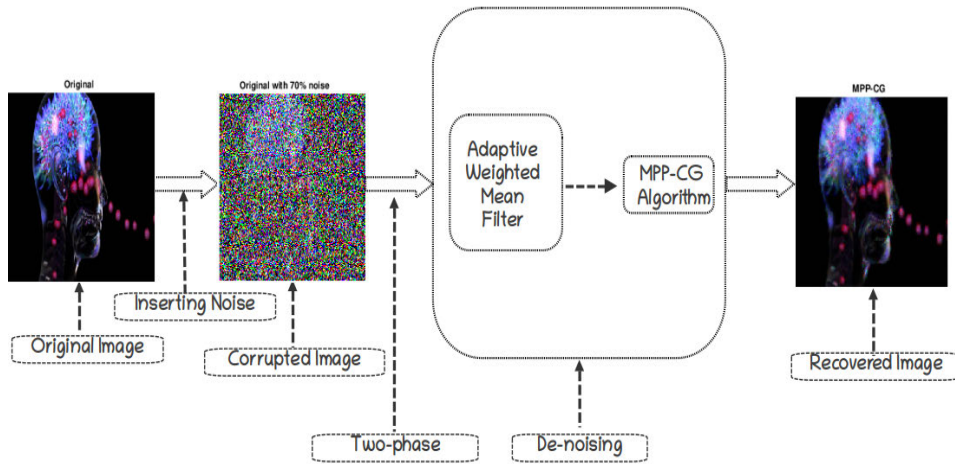


FIGURE 2. The figure shows an intuitive structure of a two-stage/two-phase method for recovery of a corrupted colour image.

Lemma 3.6: Suppose Lemma 3.4 and 3.5 hold and α^k satisfies the conditions (13) and (14). Then

$$\sum_{k=0}^{\infty} \frac{(g^{(k)T} d^{(k)})^2}{\|d^{(k)}\|^2} < +\infty. \quad (19)$$

Proof 3.7: The proof follows directly from the work of Zoutendijk [12].

Theorem 3.8: Let g denote the gradient of the objective function $\mathcal{G}_{\psi_\delta}$ and the sequence of iterates $\{\tilde{\mathbf{x}}^{(k)}\}$ be produced by Algorithm 2. Then

$$\lim_{k \rightarrow \infty} \inf \|g^{(k)}\| = 0. \quad (20)$$

Remark 3.9: The convergence analysis described above shows that Algorithm 2 is convergent for general smooth objective functions satisfying Assumption 3.2. Thus, following the analytical characteristics of the functional introduced in [11]. Theorem 2 in [11], the global convergence of

Algorithm 2 with the model objective function, $\mathcal{G}_{\psi_\delta}$ defined in (7) can be deduced. Thus, the model formulation $\mathcal{G}_{\psi_\delta}$ can be solved efficiently using MPPCG.

IV. FINE-TUNED RESTORMER

In this section, we present an efficient variant of the Transformer-based [37] architecture for the image denoising problem, specifically addressing images degraded by salt-and-pepper noise. We specifically consider a model architecture, named Restormer [36] that was initially introduced for high-resolution image processing tasks, including image deraining, single-image motion deblurring, defocus deblurring and image denoising. We first describe the innovative components of the Restormer and then diagrammatically illustrate the model pipeline.

The key important modelling innovations in Restormer are: multi-dconvolution head transposed attention (MDTA)

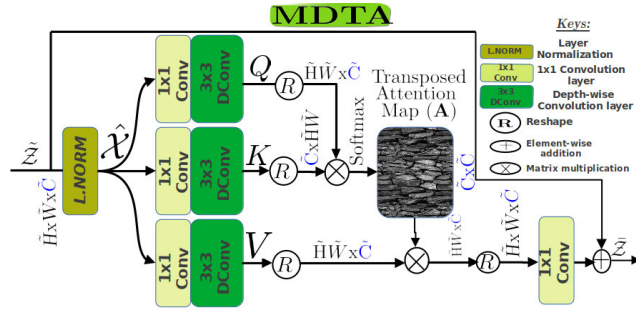


FIGURE 3. multi-dconvolution head transposed attention (MDTA).

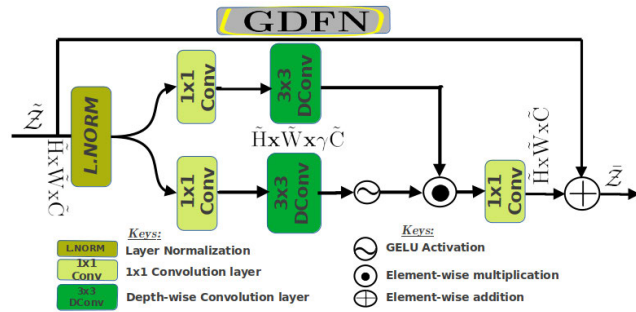


FIGURE 4. Gated-depth-wise convolution feed-forward network (GDFN).

block and gated dconvolution feed-forward network (GDFN). The MDTA block replaces the vanilla self-attention (SA) in Transformer models [38]. This SA mechanism exhibits a computational complexity that increases quadratically, thus, rendering it unsuitable for high-resolution imaging tasks. Unlike multi-head self-attention (SA) layer, where key-query multiplication scales quadratically with image input size, the MDTA possesses a linear complexity, consequently making it amenable to varying image processing problems. The reduction is achieved by computing self-attention across channels, thereby constructing an encoded global attention map through cross covariance computation [36]. It is important to note that prior attempts in complexity-reduction efforts relied on applying independent spatial patches of the input image size [39] or restricting self-attention to a small fixed-size local window [40]. The architectural design of the MDTA is illustrated in Fig. 3. The module aims to capture a local context via channel-wise feature interactions. The explicit structural equation that encapsulates the procedure of MDTA is state as follows [36]:

$$\tilde{Z} = \mathcal{W}_p^{(0)} \text{atten}(\tilde{Q}, \tilde{K}, \tilde{V}) + \tilde{Z}, \quad (21)$$

where $\tilde{Z}$ is the input feature map, $\mathcal{W}_p^{(\cdot)}$ is a pixel-wise 1×1 convolution used for assimilating spatial context in a channel-wise sense, $\text{atten}(\tilde{Q}, \tilde{K}, \tilde{V}) = \tilde{V} \cdot \text{softmax}(\tilde{K} \cdot \tilde{Q} / \lambda)$, $\tilde{Q}, \tilde{K}, \tilde{V}$ are query, key, value matrices constructed after some few operations on the normalized tensor, $\tilde{X}$, $\lambda > 0$ is a learnable scaling constant and $\tilde{Z}$ is the output to the network.

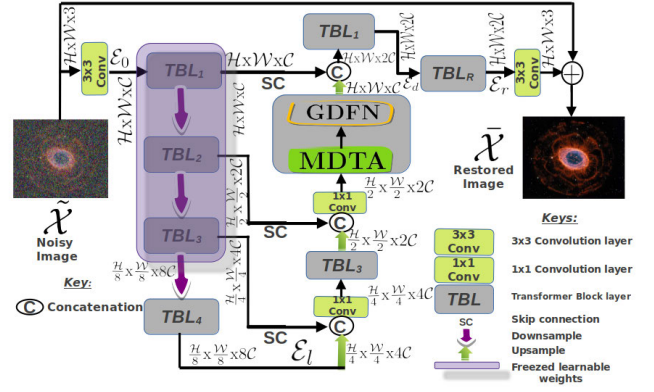


FIGURE 5. FRESTORMER model architecture.

The second key contribution is termed “gated-dconv feed-forward network” (GDFN), which serves as an information flow pathway, i.e., that enables interconnectedness, passage and preserves finer local details between layers. This module implements its information flow mechanism through a gating strategy and a depth-wise convolution layer. The latter approach acts as a feature or pattern-capturing layer by learning how nearby pixels relate to each other spatially. Moreover, the GDFN network possesses the following operational formulation given below:

$$\tilde{Z} = \mathcal{W}_p^{(0)} \text{gating}(\tilde{Z}) + \tilde{Z}, \quad (22)$$

where $\tilde{Z}$ is an input tensor, $\text{gating}(\tilde{Z})$ is an element-wise product, $\odot$ between a GELU (gaussian error linear unit) [1] nonlinearity function, ϕ and normalization layer, $\text{LN}(\cdot)$, that is, $\text{gating}(\tilde{Z}) = \phi(\mathcal{W}_d^{(1)} \mathcal{W}_p^{(1)} \text{LN}(\tilde{Z})) \odot \mathcal{W}_d^{(2)} \mathcal{W}_p^{(2)} \text{LN}(\tilde{Z})$, $\mathcal{W}_d^{(\cdot)}$ denotes a 3×3 depth-wise convolution. The network structure of GDFN is illustrated in Fig. 4.

The finetuned-restormer architecture (is simply a modified of Restormer introduced in [36]) begins by performing feature extraction via 3×3 convolution on an input noisy image represented by $\tilde{X} \in \mathbb{R}^{\mathcal{H} \times \mathcal{W} \times 3}$. The extracted feature embedding denoted by $\mathcal{E}_0$ passes through four down-sampling encoder Transformer blocks. The first three encoder blocks have frozen learnable weights, where each encoder consists of multiple Transformer blocks arranged in an increasing order of magnitude to achieve an optimize training efficiency. These encoder blocks perform progressive reduction of the spatial resolution of the feature map through down-sampling. Furthermore, the latent feature representation (i.e., the output) of the fourth encoder Transformer serves as the input to the decoder blocks.

Each of the three decoder blocks is preceded by a skip connection operation, which facilitate image resolution recovery. To ensure proper alignment of the feature spatial representation, the first two concatenation operations are followed by 1×1 convolution operations, thus effectively cutting the respective channel dimension by half without altering the spatial information. The last (topmost) decoder

TABLE 1. This gives the percentages of noise used with respective window sizes for both AMF and AWMF algorithms.

% Noise level	Max. Window size, $z_{max} \times z_{max}$	
30	7×7	[H]
60	13×13	
90	39×39	

block is preceded by a concatenation operation of an incoming skip connection and a prior decoder feature map, thus, capturing small-scale geometric image elements (such as edges, shapes, outlines, contours, thin lines e.t.c.) and surface patterns or repeated visual contents. The resulting deep feature map undergoes refinement before passing through a convolution layer that acts on the refined features, thereby yielding a restored image with an enhanced residual image. The residual image matches the dimensions of the input noisy image, thus producing the denoised output image through the summation of the residual image and the input noisy image. The encoder block levels contain 4, 6, 6 and 8 Transformer blocks, respectively, while the decoder block levels contain 6, 6, and 4 Transformer blocks, respectively.

Note that each of these encoder-decoder Transformer blocks consists of both MDTA and GDFN networks defined in (21) and (22) respectively. The schematic design of the fine-tuned restormer is described in Figure 5.

V. EXPERIMENTS

In this section of the paper, we aim to present some preliminary simulation results that showcase a glaring performance of the proposed AWMF-MPPCG algorithm compared with other methods in the literature. At this junction, it is essential to note that the AWMF-MPPCG approach is, to the best of our knowledge, the first of its kind developed purposely for restoring colored images damaged with "salt-and-pepper" noise, therefore, for a fair comparison, we extended other reported iterative algorithms to handle problem (9) together with their corresponding adaptive filter scheme as well. For the sake of brevity, in this section, we will use MPPCG to denote the AWMF-MPPCG method. Additionally, the parameters set up in these methods remain unchanged. For clarity in presentation, we partitioned the comparison into two compartments. In the first subsection, named "iterative approaches", we compare the MPPCG approach with some extended iterative methods, while in the second segment, we showcase the comparative performance of MPPCG with "non-iterative approaches".

A. ITERATIVE APPROACHES

Since the MPPCG approach consists of an iterative step in the denoising process, in this subsection, we compared it with other iterative-based methods. The methods include Algorithm 2.1 of [13] (referred to as mSGD), TPRP [14],

TABLE 2. This is the list of the images considered with their corresponding noise level used in the experiment.

S/No.	Image/Noise Level
1	brain.png/0.30 brain.png/0.60 brain.png/0.90
2	melanoma1.png/0.30 melanoma1.png/0.60 melanoma1.png/0.90
3	melanoma2.png/0.30 melanoma2.png/0.60 melanoma2.png/0.90
4	mandril.png/0.30 mandril.png/0.60 mandril.png/0.90
5	kmutt1.png/0.30 kmutt1.png/0.60 kmutt1.png/0.90
6	kmutt2.png/0.30 kmutt2.png/0.60 kmutt2.png/0.90
7	housergb.png/0.30 housergb.png/0.60 housergb.png/0.90
8	glaucoma1.png/0.30 glaucoma1.png/0.60 glaucoma1.png/0.90
9	glaucoma2.png/0.30 glaucoma2.png/0.60 glaucoma2.png/0.90

NPRP4 [15], Algorithm 0.1 [16] (referred to as mTPRP). We implemented all these algorithms, including MPPCG, on Python 3.10.12 and carried out experiments on a personal computer (PC) that possesses the following specifications: A 1.60 GHz Intel Core i5-8265U and 20 GB of RAM on Ubuntu 22.04.4 LTS Operating System. As mentioned earlier, the parameter initialization for the extended methods, namely mSGD [13], TPRP [14], NPRP4 [15], and mTPRP [16], are taken from their respective papers, while the parameters for the proposed two-phase AWMF-MPPCG are defined as follows:

- Two phase-MPPCG: In the first phase, an adaptive weighted mean filter (AWMF) is used in the *detection stage*, while MPPCG uses these parameter definitions: $\sigma = 0, 90$, $\rho = 10^{-4}$ and $tol = 10^{-6}$ in the *recovery stage*.

To effectively show the performance of these approaches, we experimented on nine(9) images of different colour intensities. These test images are obtained from varying sources which we provide as follows: http://www.imageprocessingplace.com/root_files_V3/image_databases.htm, [17], [18], while the other two images are taken from King Mongkut's University Technology Thonburi, (KMUTT's main campus <https://www.kmutt.ac.th/en/>). Additionally, we perform some simple preprocessing via normalization on the noisy pixels identified and tracked during the first filtering phase. It is worth noting that the noisy pixel, $y_{i,j,k}$ identification is carried out by checking $S_{(i,j,k)}^{min}(z) < y_{i,j,k} < S_{(i,j,k)}^{max}(z)$, where $S_{(i,j,k)}^{min}$

TABLE 3. The performance of the proposed MPPCG algorithm, in comparison with NPRP4, mSGD, TPRP, and mTPRP algorithms using Charbonnier penalty and evaluated based on metrics: ITR, PSNR, SSIM, SNR, RelErr and CPU. The best-performing approach for each metric is bold.

S/No.	MPPCG	NPRP4	mSGD
	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU
1	175/31.85/0.98/20.37/7.63/5.47 148/27.07/0.95/15.59/17/8.03 294/22.51/0.81/11.02/36.74/12.74	606/27.71/0.97/16.22/13.18/8.34 606/23.19/0.9/11.7/28.5/16.02 606/18.04/0.7/6.55/61.41/19.43	1212/28.21/0.97/16.72/12.35/15.79 808/23.58/0.91/12.1/26.85/22.05 624/18.87/0.73/7.38/56.29/20.58
2	123/38.03/0.98/33.5/0.99/4.99 113/33.95/0.95/29.42/2.1/6.96 141/30.33/0.89/25.8/4.53/8.46	606/35.27/0.97/30.74/1.3/8.84 606/31.43/0.92/26.9/2.61/15.76 606/26.24/0.76/21.71/8.18/19.28	884/36.03/0.97/31.5/1.21/12.65 734/32/0.93/27.47/2.35/20.05 1327/26.74/0.78/22.21/7.65/40.14
3	124/47.2/1/44.94/0.3/4.25 97/42.27/1/40.02/0.71/6.19 110/36.85/0.99/34.6/1.73/8.1	416/42.45/1/40.19/0.45/5.95 523/38.16/0.99/35.91/1.17/14.42 606/31.02/0.98/28.76/4.63/19.69	952/43.28/1/41.03/0.39/11.7 925/38.78/1/36.53/1.07/25.25 717/31.36/0.98/29.11/4.49/23.8
4	145/29.25/0.95/23.41/4.72/5.06 124/25.18/0.87/19.33/10.1/8.75 245/21.47/0.69/15.62/19.3/12.69	606/26.91/0.92/21.06/6.08/7.65 606/22.64/0.8/16.79/13.54/16.48 606/17.48/0.52/11.64/29.73/20.13	422/27.5/0.93/21.65/5.66/5.8 625/23.11/0.82/17.26/12.72/18.38 900/18.52/0.58/12.68/26.21/30.85
5	158/27.89/0.94/21.94/5.16/4.99 171/23.46/0.84/17.52/12.17/9.03 282/18.99/0.57/13.05/26.64/13.34	606/24.67/0.9/18.72/8.15/8.1 606/19.88/0.71/13.94/18.39/15.61 606/14.55/0.29/8.61/41.95/19.96	1244/25.13/0.91/19.19/7.65/16.02 588/20.27/0.73/14.33/17.43/16.36 765/15.39/0.35/9.45/38.34/24.65
6	154/26.15/0.94/19.84/7.6/4.94 132/22.14/0.83/15.83/16.44/7.46 292/18.66/0.6/12.35/30.29/13.29	606/23.72/0.89/17.42/10.85/7.93 606/19.74/0.73/13.43/22.35/15.79 606/15.07/0.4/8.76/44.01/19.42	848/24.22/0.91/17.91/10.25/10.83 827/20.16/0.75/13.86/21.16/22.74 943/16.15/0.48/9.84/38.92/29.55
7	150/30.08/0.98/25.46/1.51/4.74 137/28.32/0.96/23.71/3.8/7.55 223/24.52/0.89/19.91/10.61/11.75	606/28.62/0.97/24.25/7.87 606/25.27/0.92/20.66/6.64/15.64 606/19.45/0.75/14.84/17.31/19.42	498/28.83/0.97/24.21/2.39/6.61 840/25.51/0.92/20.9/6.35/22.96 1178/19.70/76/15.08/16.78/35.79
8	217/33.56/0.96/26.57/1.44/6.69 207/28.64/0.91/21.65/3.23/14.86 351/19.91/0.79/12.92/7.41/18.09	413/32.22/0.99/25.23/2.37/7.45 578/27.44/0.97/20.45/5.44/19.2 606/19.74/0.85/12.75/13.2/35.85	896/32.58/0.99/25.59/2.11/15.23 922/27.66/0.97/20.68/4.95/27.91 1048/19.89/0.86/12.9/12.39/48.1
9	512/29.38/0.91/21.88/4.69/11.64 370/21.18/0.84/13.68/10.49/14.23 605/15.5/0.74/8/17.82/22.62	534/29.51/0.99/22.01/5.43/9.32 606/22.34/0.93/14.84/10.05/17.11 606/16.42/0.78/8.92/31.66/34.04	501/29.74/0.99/22.24/5.25/8.37 821/22.46/0.93/14.96/9.77/22.49 755/16.61/0.78/9.11/30.97/37.84

TABLE 4. This is a continuation of 3 with TPRP and mTPRP algorithms using Huber loss function defined in (V-A) and evaluated based on metrics: ITR, PSNR, SSIM, SNR, RelErr and CPU. The best-performing approach for each metric is bold.

S/No.	TPRP	mTPRP
	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU
1	319/31.17/0.98/19.69/8.75/5.7 438/26.25/0.95/14.77/18.98/15.01 303/20.95/0.8/9.46/46.6/13.72	579/31.69/0.99/20.21/7.9/10.42 548/26.94/0.95/15.46/17.5/21.46 605/21.99/0.82/10.5/41.24/39.74
2	191/36.33/0.97/31.8/1.16/3.29 165/33.14/0.94/28.6/2.03/5.77 229/29.79/0.88/25.26/4.66/10.63	149/36.56/0.98/32.03/1.13/2.84 189/33.46/0.95/28.93/1.97/7.17 285/30.06/0.89/25.53/3.99/15.6
3	127/44.57/1/42.31/0.31/2.41 130/39.64/1/37.39/0.94/4.87 250/34.51/0.99/32.25/3.2/11.33	106/44.9/1/42.64/0.3/2.16 147/39.75/1/37.49/0.93/4.98 235/35.32/0.99/33.06/2.46/13.58
4	352/28.93/0.95/23.08/4.87/5.97 374/24.85/0.86/19.01/10.38/12.79 70/20.54/0.67/14.7/21.07/16.41	413/28.98/0.95/23.14/4.85/7.7 370/24.97/0.86/19.13/10.26/14.7 594/21.17/0.69/15.33/19.84/34.02
5	345/27.66/0.94/21.72/5.29/6.42 396/22.9/0.83/16.96/12.71/14.11 367/17.61/0.5/11.67/30.58/16.23	516/27.77/0.94/21.83/5.11/9.62 608/23.33/0.84/17.39/11.92/24.41 603/18.52/0.56/12.58/27.22/35.52
6	436/25.85/0.93/19.54/8.03/7.37 469/21.8/0.82/15.49/17.12/16.65 506/17.97/0.57/11.66/32.6/21.4	446/25.91/0.93/19.6/7.85/8.12 551/21.93/0.82/15.62/16.71/20.86 604/18.41/0.59/12.1/31.73/35.91
7	240/29.86/0.98/25.25/1.65/4.15 340/27.71/0.96/23.09/4.19/11.18 313/22.06/0.84/17.45/13.29/13.16	511/30.04/0.98/25.43/1.5/10.26 506/28.33/0.96/23.72/3.61/20.63 530/23.86/0.88/19.25/11.66/29.87
8	184/32.8/0.99/25.81/2.07/4.24 213/28.48/0.96/21.49/4.12/11.64 313/22.07/0.86/15.08/8.13/27.32	151/32.86/0.99/25.87/1.99/4.47 208/28.52/0.96/21.53/4.11/10.41 490/22.71/0.83/15.72/7.04/39.39
9	177/30.29/0.98/22.79/4.69/4.07 316/24.07/0.92/16.57/6.93/12.21 320/18.34/0.8/10.84/26.02/28.24	412/30.62/0.96/23.12/4.2/10.34 608/24.35/0.89/16.85/5.93/28.19 604/18.66/0.77/11.16/22.24/49.8

and $S_{(i,j,k)}^{max}$ are the computed minimum and maximum pixel values within given image window size $z \times z$. If the condition

fails, then pixel $y_{i,j,k}$ is noisy and hence, it is replaced by a median in the setting of AMF or by its weighted mean in the

TABLE 5. The performance of the proposed MPPCG algorithm, in comparison with NPRP4, mSGD, TPRP, and mTPRP algorithms using Huber loss function defined in (8) and evaluated based on metrics: ITR, PSNR, SSIM, SNR, RelErr and CPU. The best-performing approach for each metric is bold.

S/No.	MPPCG	NPRP4	mSGD
	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU
1	149/30.41/0.98/18.93/8.87/4.23 165/26.15/0.93/14.66/18.63/7.11 79/21.81/0.82/10.32/38.21/5.06	606/27.63/0.96/16.15/13.31/8.89 81/23.08/0.89/11.6/28.96/3.07 71/17.99/0.7/6.5/61.77/3.9	1091/28.5/0.97/17.02/11.95/15.34 1092/23.73/0.9/12.25/26.35/31.93 1006/18.68/0.77/2.57/69/30.76
2	90/36.8/0.98/32.27/1.13/3.97 97/33.23/0.94/28.7/2.22/5.46 150/29.83/0.87/25.3/5.16/7.37	606/35.18/0.97/30.65/1.32/8.44 606/31.39/0.92/26.86/2.64/16.85 606/26.23/0.76/21.7/8.19/19.35	1148/35.69/0.97/31.16/1.24/15.52 1149/31.83/0.92/27.3/2.42/32.74 1103/26.81/0.77/22.28/7.74/35.01
3	102/44.11/1/41.85/0.38/4.32 98/40.93/1/38.67/0.81/6.26 103/36.59/0.99/34.33/1.87/6.57	569/42.34/1/40.09/0.46/8.97 606/38/0.99/35.75/1.2/19.27 606/31/0.98/28.74/4.64/21.45	1155/42.65/1/40.39/0.44/16.83 1156/38.29/0.99/36.04/1.15/39.01 1125/31.23/0.98/28.97/4.54/38.38
4	130/28.82/0.95/22.98/4.96/4.92 154/24.86/0.86/19.01/10.46/8.12 169/21.1/0.68/15.26/19.88/9.16	606/26.9/0.92/21.06/6.09/8.55 606/22.63/0.8/16.79/13.54/17.08 606/17.48/0.52/11.63/29.73/21.38	1090/27.87/0.94/22.02/5.53/14.76 1095/23.37/0.82/17.52/12.44/32.59 1009/18.23/0.56/12.39/27.72/34.24
5	158/27.23/0.93/21.29/5.52/5.05 150/22.99/0.83/17.05/12.86/7.77 164/18.45/0.53/12.51/28.19/8.75	606/24.65/0.9/18.71/8.17/8.91 606/19.88/0.71/13.93/18.41/17.24 606/14.55/0.29/8.61/41.95/21.41	1107/25.32/0.91/19.38/7.52/15.66 1091/20.37/0.73/14.43/17.43/32.41 952/15.16/0.32/9.22/40.4/32.71
6	127/25.8/0.93/19.49/8.05/4.68 199/21.93/0.82/15.62/16.91/9.44 174/18.4/0.58/12.09/30.19/9.1	606/23.72/0.89/17.42/10.85/8.66 606/19.74/0.73/13.43/22.35/17.22 606/15.07/0.4/8.76/44.03/21.43	1048/24.59/0.91/18.28/9.9/14.59 1022/20.38/0.76/14.07/20.74/30.46 1007/15.81/0.44/9.5/41.01/34.39
7	170/29.61/0.98/24.99/1.85/4.92 221/27.71/0.95/23.09/4.33/9.34 99/23.67/0.87/19.06/11.37/6.19	606/28.58/0.97/23.97/2.57/8.64 606/25.25/0.92/20.63/6.67/17.22 606/19.45/0.75/14.83/17.32/21.65	1151/28.74/0.97/24.12/2.46/15.68 1131/25.45/0.92/20.83/6.44/33.83 1112/19.71/0.76/15.1/16.86/37.7
8	161/30.32/0.95/23.33/1.56/5.94 110/26.25/0.92/19.26/3.89/7.57 83/18.28/0.8/11.29/8.26/7.33	556/32.19/0.99/25.2/2.39/10.18 606/27.39/0.97/20.41/5.56/19 606/19.73/0.85/12.74/13.31/37.03	1165/32.42/0.99/25.43/2.21/20.28 1149/27.58/0.97/20.59/5.15/34.08 1102/19.92/0.85/12.93/12.52/52.76
9	110/24.39/0.92/16.89/10.09/5.36 123/19.46/0.88/11.96/13.16/7.77 105/14.24/0.74/6.74/21.98/8.23	139/29.4/0.99/21.95/49/3.57 606/22.31/0.93/14.81/10.11/19.33 606/16.41/0.77/8.91/31.7/37.47	1176/29.68/0.99/22.18/5.32/21.22 1138/22.45/0.93/14.95/9.79/34.52 1062/16.65/0.78/9.15/31.09/52.77

TABLE 6. This is a continuation of 5 with TPRP and mTPRP algorithms using Huber loss function defined in (8) and evaluated based on metrics: ITR, PSNR, SSIM, SNR, RelErr and CPU. The best-performing approach for each metric is bold.

S/No.	TPRP	mTPRP
	ITR/PSNR/SSIM/SNR/RelErr/CPU	ITR/PSNR/SSIM/SNR/RelErr/CPU
1	276/28.47/0.97/16.98/12.04/4.32 187/24.31/0.91/12.83/24.93/7.6 188/19.84/0.71/8.36/51.63/7.56	390/28.85/0.97/17.37/11.49/5.94 440/24.33/0.91/12.85/24.83/14.07 375/19.97/0.72/8.49/50.6/14.91
2	198/35.54/0.97/31.01/1.27/3.24 230/31.83/0.93/27.3/2.43/7.29 295/26.96/0.79/22.43/7.52/11.35	255/35.58/0.97/31.04/1.27/4.37 404/31.97/0.93/27.44/2.37/12.29 475/27.44/0.8/22.91/7.15/19.66
3	205/42.59/1/40.33/0.44/3.77 251/38.28/0.99/36.03/1.15/8.44 200/31.31/0.98/29.05/4.51/8.73	206/42.64/1/40.38/0.44/3.81 433/38.37/0.99/36.12/1.13/13.7 201/31.3/0.98/29.05/4.51/8.95
4	205/28.01/0.94/22.17/5.43/3.68 272/23.75/0.83/17.9/11.92/9.04 158/19.07/0.6/13.23/25.35/7.57	322/28.16/0.94/22.31/5.36/5.74 388/24.08/0.84/18.23/11.52/12.51 340/19.38/0.61/13.54/24.58/14.12
5	186/25.55/0.91/19.61/7.36/3.53 385/20.76/0.75/14.82/16.8/13.19 194/16.43/0.39/10.48/36.31/8.72	286/25.83/0.92/19.89/7.17/5.18 397/20.94/0.75/15/16.54/13.34 341/16.56/0.4/10.62/35.68/14.14
6	186/24.95/0.92/18.64/9.41/3.47 201/20.91/0.78/14.6/19.43/7.07 194/16.86/0.5/10.55/36.72/8.65	251/25.12/0.92/18.81/9.19/4.45 355/21.1/0.79/14.79/18.95/12.27 343/16.96/0.51/10.65/36.14/14.24
7	193/28.75/0.97/24.13/2.46/3.52 226/25.49/0.92/20.87/6.41/7.41 197/20.05/0.77/15.44/16.37/8.74	516/28.82/0.97/24.21/2.47/9.99 516/25.77/0.93/21.15/6.15/16.21 394/20.43/0.78/15.82/15.87/16.05
8	232/32.4/0.99/25.41/2.22/5.12 216/27.64/0.97/20.65/5.13/8.45 211/20.15/0.85/13.16/12.13/24.62	304/32.42/0.99/25.43/2.2/6.39 464/27.7/0.97/20.71/5.07/15.96 429/20.24/0.85/13.25/11.76/32.01
9	214/29.67/0.99/22.17/5.29/4.93 435/22.51/0.93/15.01/9.73/15.46 197/16.93/0.78/9.43/30.43/26.54	391/29.7/0.99/22.2/5.26/8.31 450/22.6/0.93/15.1/9.59/16.27 380/17.11/0.78/9.61/30.03/31.58

case of AWMF; otherwise, it remains unchanged. Additionally, a noisy pixel value that undergoes either of these filtering

process is also tracked. At the second stage of the experiment (i.e., a stage where the problem (6)) is minimized, we used

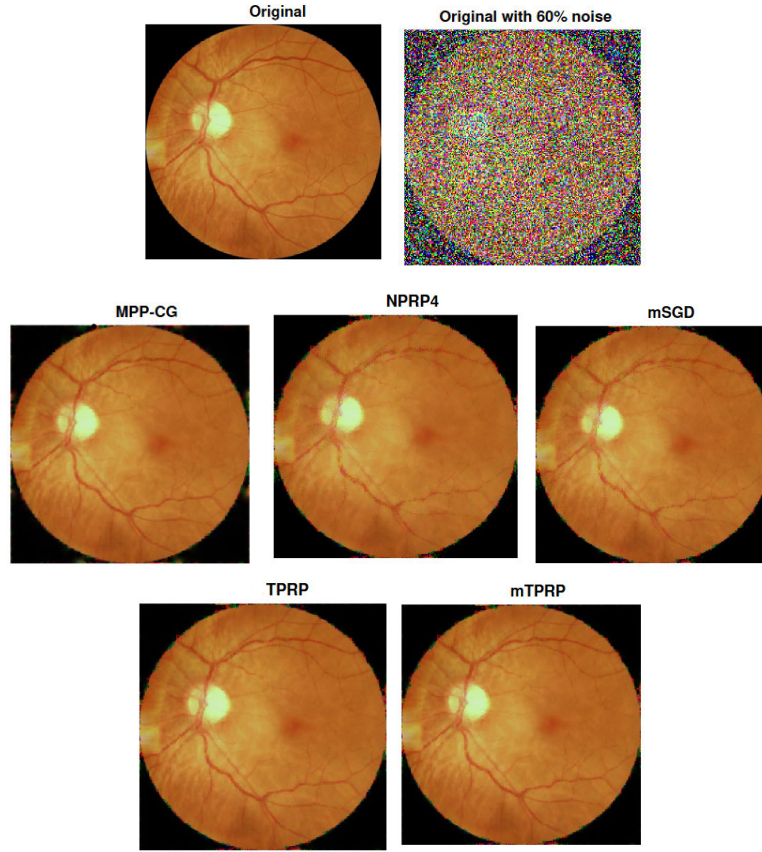


FIGURE 6. A visual display of the comparative results between the proposed MPPCG method and other methods. We used the Charbonnier edge-preserving function with 60% noise ratio on melanoma 1 image. The first row from left to right consists of the original image and its noisy variant. The second and third rows are the recovered images obtained using MPPCG, NPRP4, mSGD, TPRP and mTPRP, respectively.

the following stopping criteria

$$|\mathcal{G}_{\psi_{\delta}}(\tilde{\mathbf{x}}^{(k)}) - \mathcal{G}_{\psi_{\delta}}(\tilde{\mathbf{x}}^{(k-1)})| < tol * |\mathcal{G}_{\psi_{\delta}}(\tilde{\mathbf{x}}^{(k)})|, \quad (23)$$

where $tol = 10^{-6}$ and $\tilde{\mathbf{x}}^{(k)}$ is a denoised image at iteration k .

We characterized the experiment into two components based on the edge-preserving functional adopted. These functions are:

- (1.) Charbonnier penalty [35] $\psi_{\delta}(\tilde{x}) := \sqrt{\tilde{x}^2 + \delta}$
- (2.) A Huber loss function [10]

$$\psi_{\delta}(\tilde{x}) := \begin{cases} \frac{\tilde{x}^2}{2} & \text{for } |\tilde{x}| \leq \delta \\ \delta(|\tilde{x}| - \frac{1}{2}\delta) & \text{for } |\tilde{x}| > \delta, \end{cases} \quad (24)$$

in which the constant, $\delta = 10$ [11] is used for (1.), while $\delta = 85$ is used in (2.)

In each of the two simulations conducted, we used six (6) comparison metrics to verify or measure the effectiveness of the approaches to restore a coloured image corrupted with salt-and-pepper noise. These metrics include the number of iterations (ITR), peak signal-to-noise ratio (PSNR) (measured in decibels(dB)), structural similarity index measure (SSIM) [47], [48], signal-to-noise ratio (SNR) (measured in decibels(dB)), relative error (RelError) and CPU time. The

TABLE 7. A summarized details of the training set up.

component	detail
data set	Flickr2K
input channels	3 (RGB)
# training images	1,855
patch size	256×256
batch size	2
learning rate	10^{-4}
data augmentation	random flips
optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
loss function	ℓ_1 -loss
epoch	20
implementation framework	PyTorch 11.7

formulation of some of these metrics, for instance, PSNR and RelError, are defined as follows:

$$\text{PSNR} := 10 \log_{10} \frac{255^2}{\frac{1}{m*n*c} \|\mathbf{x} - \tilde{\mathbf{x}}^*\|_F^2},$$

$$\text{RelError} := 100 \frac{\|\mathbf{x} - \tilde{\mathbf{x}}^*\|_F}{\|\mathbf{x}\|_F},$$

where $\|\cdot\|_F$ denotes a Frobenius norm, while $\mathbf{x}$ and $\mathbf{x}^*$ represent the original and restored images respectively, $m \times n \times c$ is the size of the image and c is the number of channels.

The metrics for an algorithm are evaluated after the whole denoising procedure for a particular image is concluded.

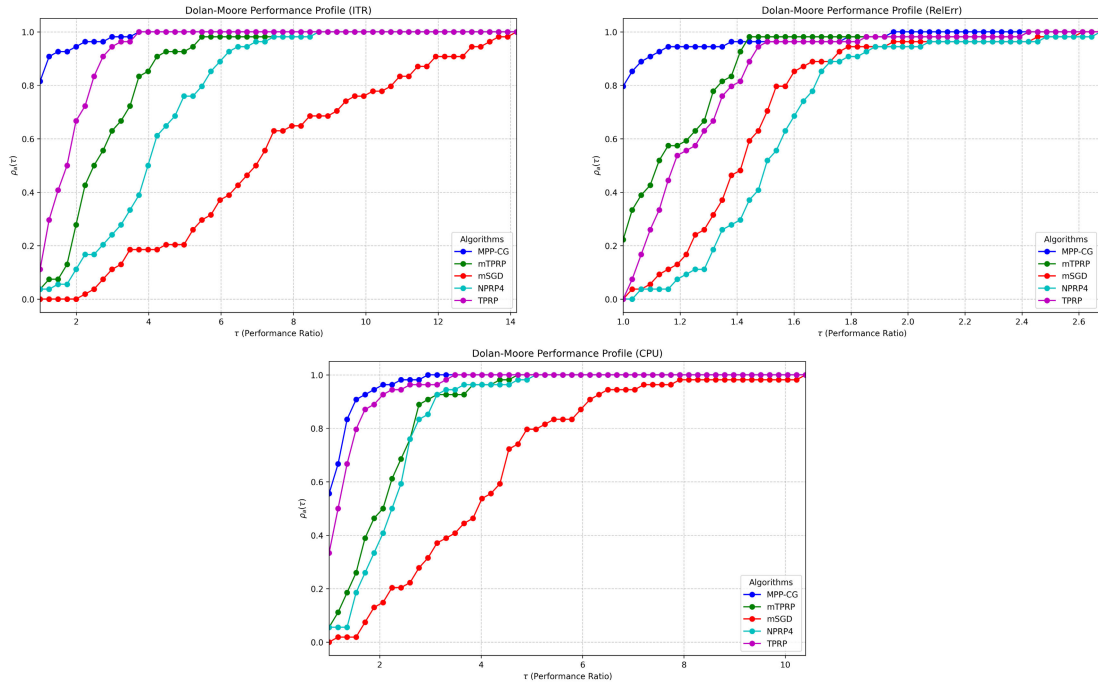


FIGURE 7. The plots show the performance of MPPCG in comparison with four(4) other algorithms using dolan and Moré [27] using number of iteration(ITR), relative error (RelErr) and computing time (CPU) respectively. The y-axis represents the % percentage of success recorded by an algorithm, while the x-axis denotes the performance ratio. We can notice that the MPPCG outperforms the rest in all three metrics. Thus, MPPCG is more effective than other compared algorithms.

TABLE 8. We reported an average performance of VMF, MPPCG and FRESTORMER methods on the test images. The score is reported for the three noise levels: 30%, 60% and 90% respectively. We used Mthds, NL, and Mtrc to denote methods, noise level, and metrics respectively. Moreover, we used symbols * and ** to represent that for FRESTORMER, the inference time is measured in CPU and GPU respectively. The best-performing approach for each metric is bold.

Mthd \ Mtrc	NL		
	0.3	0.6	0.9
	PSNR/SSIM/RelErr/CPU	PSNR/SSIM/RelErr/CPU	PSNR/SSIM/RelErr/CPU
VMF	33.07/0.69/1.1409/ 0.8867	32.14/0.61/1.1615/ 0.8771	28.94/0.04/ 1.0715/0.8866
MPPCG	38.25/0.94/ 0.5954 /5.7533	35.20/0.88/ 0.8373 /12.8975	31.85/0.68/1.1640/20.3889
FRESTORMER	41.21/0.95 /0.8568/10.1531* 0.0462**	36.91/0.89 /1.0771/10.2857* 0.0442**	32.71/0.74 /1.2350/10.3342* 0.0442**

In these experiments, we consider the following configurations of the percentages of noise and maximum window size used in the filter-based stage for each image as shown in Table 1 [6].

The results obtained from experimenting with Charbonnier penalty and Huber penalty are illustrated in Tables 3 and 5, respectively. Moreover, we also show the visual performance output of one of the images in Figures 6a-6g with 60% using a window size of 17×17 .

To better illustrate the comparative performance of these algorithms, we **bold** any metric number corresponding to a best-performing algorithm for a considered image under each noise level. A quick glance at the tabular data in 3 and 5 indicates that MPPCG performs better than NPRP4 [15], mSGD [13], TPRP [14], and mTPRP [16] with respect to the comparative metrics in almost all the images used in the experiments. However, there are a few exceptions, for instance in glaucoma images where the proposed MPPCG is competitive with the mTPRP algorithm. Thus, overall, the MPPCG approach gives encouraging results on a large proportion of images considered. This is evident from the

higher values of PSNR, SSIM, and SNR recorded by MPPCG. We reported an averaged result under PSNR, SSIM and SNR metrics, respectively, using bar plots (see Figure 8). Moreover, the performances with respect to the number of iterations (ITR), CPU time and relative error rate, RelErr are also summarized using Dolan and Moré [27] performance profile (see Figure 7), where a method with the least ITR, smallest CPU and minimum RelErr is represented by a topmost curve on the y-axis of the plots. Upon viewing these plots, we can see that the MPPCG outperformed other existing methods. Therefore, an alternative approach that is efficient and robust for denoising a noisy image corrupted by salt-and-pepper noise is developed.

B. NON-ITERATIVE APPROACHES

In this subsection, we compare MPPCG with methods that we referred to as non-iterative. These are approaches for colour image denoising that do not possess or use the dual-phase steps described in Section III. In particular, we consider two approaches, namely: vector median filter (VMF) [28] and a finetuned restormer presented in Section IV. We will use

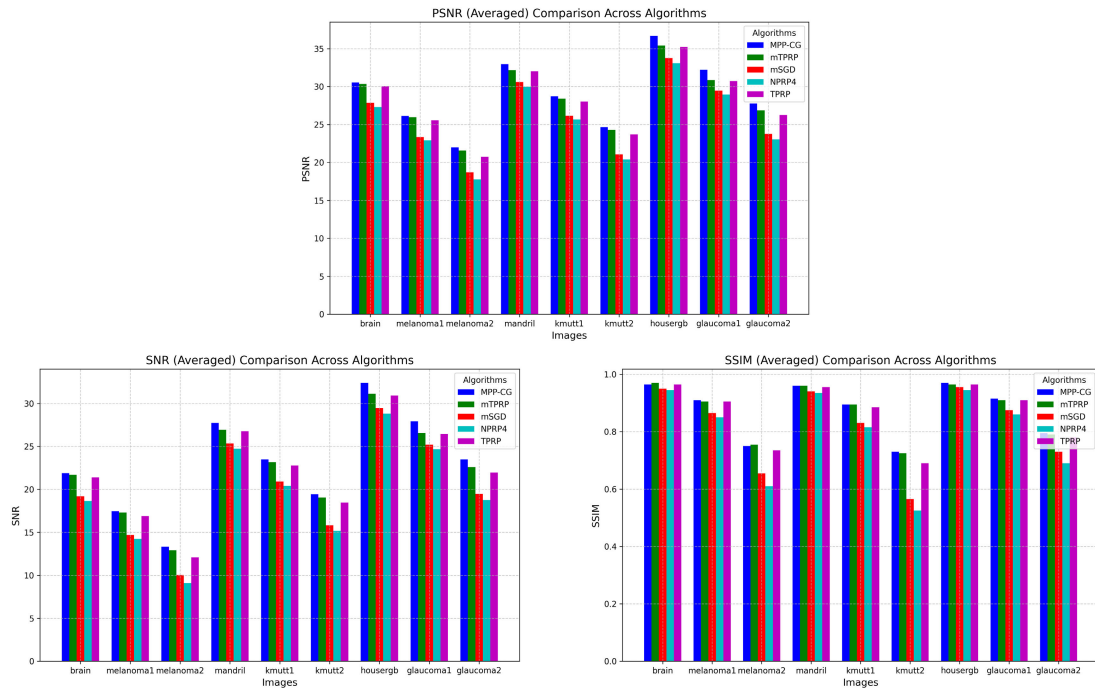


FIGURE 8. The plots illustrate the performance of MPPCG compared to four other algorithms, presenting the average results from the experiments using the two edge-preserving functionals. The y-axis represents the average values for three metrics (PSNR, SNR and SSIM), while the x-axis corresponds to the images used in the test.

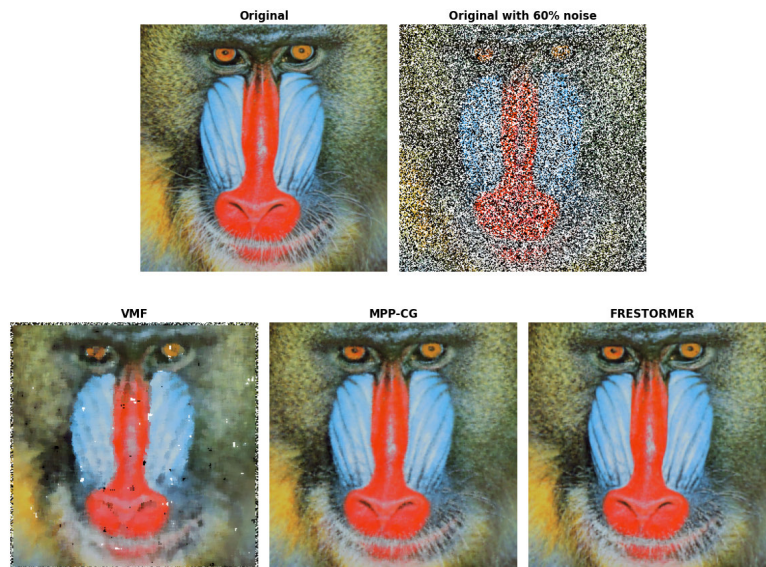


FIGURE 9. A visual comparison that portrayed the denoising performance of the VMF, MPPCG and FRESTORMER on a corrupted image with 60% noise level.

FRESTORMER to denote the finetuned RESTORMER model. Next, we describe the dataset set-up used in training and evaluating FRESTORMER and its implementation details.

1) DATASET

Following the approach in [36], we use high-resolution images from Flickr2K dataset which is accessible via <https://www.kaggle.com/datasets/daehoyang/flickr2k>, which contains +2,650+ images in total. To finetune the model,

we used 1,855 accounting for 70% of the total image set, where for each of these images, we generated 10 randomly corrupted samples with salt-and-pepper noise, ε . To ensure that the model learns efficiently while avoiding learning any specific noise ratio and thus potentially generalizes pretty well, the noise $\varepsilon \sim \text{Uniform}([0.05, 0.95] \setminus \{0.3, 0.6, 0.9\})$, i.e., we excluded the following set $\{0.3, 0.6, 0.9\}$ of noise ratios in the generated training set. The excluded set, i.e., $\{0.3, 0.6, 0.9\}$, is used in generating the corrupted samples

used during testing. Therefore, the total number of training samples becomes 18,550 while the testing and validation sample counts are 1,197 and 1,191, respectively.

2) IMPLEMENTATION

We start up the finetuning by leveraging the pre-trained weights of the Restormer model that was trained with Gaussian noise. As illustrated in Figure 5, we froze the first three encoder blocks, which often serve as feature extractors containing embedded information about image edges and textures, and may also possess pre-existing learned representations for denoising noisy images. We adopted AdamW [49] algorithm in the model training with hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.999$ and the learning rate is set to 10^{-4} . Additionally, we trained FRESTORMER using ℓ_1 -loss function. The codes for FRESTORMER are written on PyTorch 11.7 [50], and the training is conducted on a graphical processing unit, GPU-capable machine (NVIDIA GeForce RTX 4060). The model weights are accessible through this link: <https://github.com/MAHMOUDPD/image-denoising>. A summary of the relevant implementation details is presented in Table 7 for more clarity.

3) RESULTS

We discuss the results obtained from VMF, MPFCG and FRESTORMER methods, respectively. The evaluation of these methods is conducted on the noisy test images, and those on Table 2. We consider four metrics: PSNR, SSIM, RelErr and CPU (seconds) in the comparison, for each of the three levels of noise—30%, 60% and 90%. The average performance metrics with respect to each method are presented in 8. The averages are computed over each of the three investigated noisy ratios. In terms of PSNR and SSIM metrics, respectively, the FRESTORMER outperforms the MPFCG and VMF on average. The closely followed method is MPFCG. Additionally, we illustrate visually in Fig. 9 the denoising performance of these methods using one of the tested images corrupted with a 60% level of noise. We can easily see that the FRESTORMER model and MPFCG effectively denoised the corrupted image, which is clearly in contrast to the VMF method.

VI. CONCLUSION

This article extended the adaptive weighted mean filter (AWMF), initially introduced in [7] for colour image salt-and-pepper denoising. Moreover, we also introduced an extended regularization functional for colour images in the two-stage salt-and-pepper noise removal. In its initial formulation, the edge-preserving function used in the *noise removal stage* was first put forward to tackle only grey images; we extended the formulation to colour images. On the other hand, even for grey-scale images, the usual adaptive filter primarily used in the first stage, often called *noise detection stage* is an adaptive median filter (AMF) (see, for example, in [15], [13], [14], and [16]), we used AWMF which reduces the detection errors by replacing a noisy candidate with

weighted mean instead of a median as used in AMF and also AWMF aid in reducing some loss of important image details. Moreover, we also developed an efficient iterative algorithm in a framework of conjugate gradient methods, MPFCG, for minimizing the extended formulation (7) whose global convergence we established under some mild assumptions. The experiments showed that the developed MPFCG approach offers a better qualitative recovery that is close to the original images, even with a higher proportion of noise levels than other introduced approaches in the literature. Moreover, we investigate the performance of MPFCG and the data-driven method. In particular, we consider a transformer-based architecture tailored for the denoising task. In future, it will be interesting to modify or extend the MPFCG to handle other forms of noise, such as impulse-Gaussian noise and random-valued noise. One can also explore a potential hybridization with deep-learning approaches such as those presented in [29], [30], and [31], for other forms of image processing problems like image-dehazing problems.

APPENDIX

Proof 6.1: of Lemma 3.4

The proof of (18) is established using a mathematical induction technique. It is obvious that when $k = 0$, we have from Step 0 of Algorithm 2, that $g^{(0)T}d^{(0)} \leq -\|g^{(0)}\|^2 \leq -m\|g^{(0)}\|^2$. Thus, the inequality (18) holds for $k = 0$. Now, for the induction step, assume that the inequality (18) holds for $k - 1$. Then we obtain

$$g^{(k-1)T}d^{(k-1)} \leq -m\|g^{(k-1)}\|^2. \quad (25)$$

Next, we aim to show that the conclusion of (13) holds for k . Now, from the curvature condition (14), we can easily deduce that

$$\rho d^{(k-1)T}g^{(k)} - d^{(k-1)T}g^{(k-1)} \geq (\sigma\rho - 1)d^{(k-1)T}g^{(k-1)}. \quad (26)$$

To fulfill the aim, we consider two cases.

Case 1: Suppose

$$\max \left\{ \rho d^{(k-1)T}g^{(k)} - g^{(k-1)T}d^{(k-1)} + 1/\alpha^k, \|g^{(k)}\| \|d^{(k-1)}\| \right\} = \rho d^{(k-1)T}g^{(k)} - g^{(k-1)T}d^{(k-1)} + 1/\alpha^k$$

then

$$\begin{aligned} g^{(k)T}d^{(k)} &= -g^{(k)T}g^{(k)} + \beta^{(k)}g^{(k)T}d^{(k-1)} \\ &\leq -\|g^{(k)}\|^2 \\ &\quad + \frac{\rho\|g^{(k)}\|^2}{\rho d^{(k-1)T}g^{(k)} - g^{(k-1)T}d^{(k-1)} + 1/\alpha^k} g^{(k)T}d^{(k-1)} \\ &\leq \|g^{(k)}\|^2 \left(\frac{g^{(k-1)T}d^{(k-1)}}{\rho d^{(k-1)T}g^{(k)} - g^{(k-1)T}d^{(k-1)} + 1/\alpha^k} \right) \\ &\leq \|g^{(k)}\|^2 \left(\frac{g^{(k-1)T}d^{(k-1)}}{(\sigma\rho - 1)g^{(k-1)T}d^{(k-1)}} \right) \\ &= -\frac{1}{(1 - \sigma\rho)} \|g^{(k)}\|^2 \\ &\leq -(1 - \rho)\|g^{(k)}\|^2. \end{aligned}$$

Case 2: Suppose, $\max\{\rho d^{(k-1)T} g^{(k)} - g^{(k-1)T} d^{(k-1)} + 1/\alpha^k, \|g^{(k)}\| \|d^{(k-1)}\|\} = \|g^{(k)}\| \|d^{(k-1)}\|$, then

$$\begin{aligned} g^{(k)T} d^{(k)} &= -g^{(k)T} g^{(k)} + \beta^{(k)} g^{(k)T} d^{(k-1)} \\ &= -\|g^{(k)}\|^2 + \frac{\rho \|g^{(k)}\|^2}{\|g^{(k)}\| \|d^{(k-1)}\|} g^{(k)T} d^{(k-1)} \\ &\leq -\|g^{(k)}\|^2 + \frac{\rho \|g^{(k)}\|^2}{\|g^{(k)}\| \|d^{(k-1)}\|} |g^{(k)T} d^{(k-1)}| \\ &\leq -\|g^{(k)}\|^2 + \frac{\rho \|g^{(k)}\|^2}{\|g^{(k)}\| \|d^{(k-1)}\|} \|g^{(k)}\| \|d^{(k-1)}\| \\ &= -(1 - \rho) \|g^{(k)}\|^2. \end{aligned}$$

Thus, we have the required result.

Proof 6.2: of Lemma 3.5

From the definition of (14), we can have,

$$\begin{aligned} \|d^{(k)}\| &\leq \|g^{(k)}\| + |\beta^{(k)}| \|d^{(k-1)}\| \\ &\leq \|g^{(k)}\| + \frac{\rho \|g^{(k)}\|^2}{\|g^{(k)}\| \|d^{(k-1)}\|} \|d^{(k-1)}\| \\ &= (1 + \rho) \|g^{(k)}\| \\ &\leq (1 + \rho) h_1 := \widehat{b}. \end{aligned}$$

Hence, the proof is complete.

Proof 6.3: of Theorem 3.8

Suppose, by contradiction, (20) fails to be satisfied, then we can find a positive constant $H > 0$ such that

$$\|g^{(k)}\| \geq H, \quad k \geq 0. \quad (27)$$

Now, by squaring both sides of (18) we have

$$(g^{(k)T} d^{(k)})^2 \geq m^2 \|g^{(k)}\|^4. \quad (28)$$

Next, dividing both sides of (28) by $\|d^{(k)}\|^2$ and summing, we get

$$\begin{aligned} \sum_{k=0}^{\infty} \frac{(g^{(k)T} d^{(k)})^2}{\|d^{(k)}\|^2} &\geq m^2 \sum_{k=0}^{\infty} \frac{\|g^{(k)}\|^4}{\|d^{(k)}\|^2} \\ &\geq m^2 \sum_{k=0}^{\infty} \frac{H^4}{\widehat{b}^2} \\ &= +\infty. \end{aligned}$$

This contradicts the established inequality (19). Thus, (20) holds.

REFERENCES

- [1] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," 2016, *arXiv:1606.08415*.
- [2] G. Yu, J. Huang, and Y. Zhou, "A descent spectral conjugate gradient method for impulse noise removal," *Appl. Math. Lett.*, vol. 23, no. 5, pp. 555–560, May 2010.
- [3] E. Polak and G. Ribiere, "Note sur la convergence de méthodes de directions conjuguées," *Revue Française d'Informatique et de Recherche Opérationnelle. Série Rouge*, vol. 3, no. 16, pp. 35–43, 1969.
- [4] R. Fletcher, *Practical Methods of Optimization*. Wiley, 2013.
- [5] J. Barzilai and J. M. Borwein, "Two-point step size gradient methods," *IMA J. Numer. Anal.*, vol. 8, no. 1, pp. 141–148, 1988.
- [6] R. H. Chan and M. Nikolova, "Salt-and-pepper noise removal by median-type noise detectors and detail-preserving regularization," *IEEE Trans. Image Process.*, vol. 14, no. 10, pp. 1479–1485, Oct. 2005.
- [7] P. Zhang and F. Li, "A new adaptive weighted mean filter for removing salt-and-pepper noise," *IEEE Signal Process. Lett.*, vol. 21, no. 10, pp. 1280–1283, Oct. 2014.
- [8] M. Nikolova, "A variational approach to remove outliers and impulse noise," *J. Math. Imag. Vis.*, vol. 20, nos. 1–2, pp. 99–120, Jan. 2004.
- [9] R. H. Chan, C.-W. Ho, C.-Y. Leung, and M. Nikolova, "Minimization of detail-preserving regularization functional by Newton's method with continuation," in *Proc. IEEE Int. Conf. Image Process.*, Sep. 2005, pp. 1–125.
- [10] P. J. Huber, "Robust estimation of a location parameter," *Ann. Math. Statist.*, vol. 35, no. 1, pp. 73–101, Mar. 1964. [Online]. Available: <http://www.jstor.org/stable/2238020>
- [11] J.-F. Cai, R. H. Chan, and C. Di Fiore, "Minimization of a detail-preserving regularization functional for impulse noise removal," *J. Math. Imag. Vis.*, vol. 29, no. 1, pp. 79–91, Sep. 2007.
- [12] G. Zoutendijk, "Nonlinear programming, computational methods," in *Integer and Nonlinear Programming*, J. Abadie, Ed., Amsterdam, The Netherlands: North-Holland, 1970, pp. 37–86.
- [13] J. Liu and S. Li, "Spectral gradient method for impulse noise removal," *Optim. Lett.*, vol. 9, no. 7, pp. 1341–1351, Oct. 2015.
- [14] J. Liu, H. Cao, Y. Zhao, and L. Zhang, "A gradient-type iterative method for impulse noise removal," *Numer. Linear Algebra With Appl.*, vol. 28, no. 4, pp. e235–8, Aug. 2021.
- [15] A. Mousavi, M. Esmailpour, and A. Sheikahmadi, "A new family of Polak–Ribière–Polyak conjugate gradient method for impulse noise removal," *Soft Comput.*, vol. 27, pp. 17515–17524, Dec. 2023.
- [16] P. Gao and Y. Wu, "A sufficient descent conjugate gradient method for impulse noise removal," *ScienceAsia*, vol. 50, no. 1, p. 1, 2024.
- [17] R. Kiefer, M. Abid, J. Steen, M. R. Ardali, and E. Amjadian, "A catalog of public glaucoma datasets for machine learning applications: A detailed description and analysis of public glaucoma datasets available to machine learning engineers tackling glaucoma-related problems using retinal fundus images and OCT images," in *Proc. 7th Int. Conf. Inf. Syst. Data Mining (ICISDM)*, May 2023, pp. 24–31.
- [18] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Sci. Data*, vol. 5, no. 1, pp. 1–9, Aug. 2018.
- [19] T. Huang, G. Yang, and G. Tang, "A fast two-dimensional median filtering algorithm," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. ASSP-27, no. 1, pp. 13–18, Feb. 1979.
- [20] H. Hwang and R. A. Haddad, "Adaptive median filters: New algorithms and results," *IEEE Trans. Image Process.*, vol. IP-4, no. 4, pp. 499–502, Apr. 1995.
- [21] H.-L. Eng and K.-K. Ma, "Noise adaptive soft-switching median filter," *IEEE Trans. Image Process.*, vol. 10, no. 2, pp. 242–251, Feb. 2001.
- [22] S. Zhang and M. A. Karim, "A new impulse detector for switching median filters," *IEEE Signal Process. Lett.*, vol. 9, no. 11, pp. 360–363, Nov. 2002.
- [23] Z. Wang and D. Zhang, "Progressive switching median filter for the removal of impulse noise from highly corrupted images," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 46, no. 1, pp. 78–80, Aug. 1999.
- [24] R. Fletcher and M. J. D. Powell, "A rapidly convergent descent method for minimization," *Comput. J.*, vol. 6, no. 2, pp. 163–168, Aug. 1963.
- [25] W. Cheng, "A two-term PRP-based descent method," *Numer. Funct. Anal. Optim.*, vol. 28, nos. 11–12, pp. 1217–1230, Dec. 2007.
- [26] J. V. Burke and A. Engle, "Line search and trust-region methods for convex-composite optimization," 2018, *arXiv:1806.05218*.
- [27] E. D. Dolan and J. J. Moré, "Benchmarking optimization software with performance profiles," *Math. Program.*, vol. 91, no. 2, pp. 201–213, Jan. 2002.
- [28] J. Astola, P. Haavisto, and Y. Neuvo, "Vector median filters," *Proc. IEEE*, vol. 78, no. 4, pp. 678–689, 1990.
- [29] S. Zhang, X. Zhang, S. Wan, W. Ren, L. Zhao, and L. Shen, "Generative adversarial and self-supervised dehazing network," *IEEE Trans. Ind. Informat.*, vol. 20, no. 3, pp. 4187–4197, Mar. 2024.
- [30] S. Zhang, W. Ren, X. Tan, Z.-J. Wang, Y. Liu, J. Zhang, X. Zhang, and X. Cao, "Semantic-aware dehazing network with adaptive feature fusion," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 454–467, Jan. 2023.

- [31] Y. Liu, Z. Yan, T. Ye, A. Wu, and Y. Li, "Single nighttime image dehazing based on unified variational decomposition model and multi-scale contrast enhancement," *Eng. Appl. Artif. Intell.*, vol. 116, Nov. 2022, Art. no. 105373.
- [32] J. Xu, L. Wang, and Z. Shi, "A switching weighted vector median filter based on edge detection," *Signal Process.*, vol. 98, pp. 359–369, May 2014.
- [33] L. Jin and D. Li, "A switching vector median filter based on the CIELAB color space for color image restoration," *Signal Process.*, vol. 87, no. 6, pp. 1345–1354, Jun. 2007.
- [34] S.-J. Ko and Y. H. Lee, "Center weighted median filters and their applications to image enhancement," *IEEE Trans. Circuits Syst.*, vol. 38, no. 9, pp. 984–993, Aug. 1991.
- [35] A. Bruhn, J. Weickert, and C. Schnörr, "Lucas/Kanade meets Horn/Schunck: Combining local and global optic flow methods," *Int. J. Comput. Vis.*, vol. 61, no. 3, pp. 211–231, Feb. 2005.
- [36] S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, and M. Yang, "Restormer: Efficient transformer for high-resolution image restoration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 5718–5729.
- [37] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, and A. N. Gomez, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [38] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, and S. Gelly, "An image is worth 16 × 16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.
- [39] H. Chen, Y. Wang, T. Guo, C. Xu, Y. Deng, Z. Liu, S. Ma, C. Xu, C. Xu, and W. Gao, "Pre-trained image processing transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12294–12305.
- [40] Z. Wang, X. Cun, J. Bao, W. Zhou, J. Liu, and H. Li, "A general U-shaped transformer for image restoration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2021, pp. 17683–17693.
- [41] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.
- [42] C. Tian, Y. Xu, and W. Zuo, "Image denoising using deep CNN with batch renormalization," *Neural Netw.*, vol. 121, pp. 461–473, Jan. 2020.
- [43] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, Jul. 2017.
- [44] K. Zhang, W. Zuo, S. Gu, and L. Zhang, "Learning deep CNN denoiser prior for image restoration," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2808–2817.
- [45] K. Zhang, W. Zuo, and L. Zhang, "FFDNet: Toward a fast and flexible solution for CNN-based image denoising," *IEEE Trans. Image Process.*, vol. 27, no. 9, pp. 4608–4622, Sep. 2018.
- [46] J. Liu, J. Lin, X. Li, W. Zhou, S. Liu, and Z. Chen, "LIRA: Life-long image restoration from unknown blended distortions," in *Proc. ECCV*. Cham, Switzerland: Springer, 2020, pp. 616–632.
- [47] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: [10.1109/TIP.2003.819861](https://doi.org/10.1109/TIP.2003.819861). [Online]. Available: <https://ece.uwaterloo.ca/~z70wang/publications/ssim.pdf>
- [48] A. N. Avanaki, "Exact global histogram specification optimized for structural similarity," *Opt. Rev.*, vol. 16, no. 6, pp. 613–621, Nov. 2009, doi: [10.1007/s10043-009-0119-z](https://doi.org/10.1007/s10043-009-0119-z).
- [49] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent.*, 2017.
- [50] A. Paszke, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.



MAHMOUD M. YAHAYA received the B.S. degree in pure/applied mathematics from Gombe State University, Gombe, in September 2016, and the master's and Ph.D. degrees in applied mathematics from the King Mongkut's University of Technology, Thonburi, in 2022 and 2025, respectively. His research areas centred around convex/non-convex optimization algorithms, stochastic optimization, nonlinear least-squares methods, and applied machine learning/deep learning approaches.



POOM KUMM (Senior Member, IEEE) received the M.Sc. degree in mathematics from Chiang Mai University, in 2002, and the Ph.D. degree in mathematics from Naresuan University, Thailand, in 2007. He is currently a Full Professor with the Department of Mathematics, King Mongkut's University of Technology Thonburi (KMUTT). He is the Director of the Center of Excellence in Theoretical and Computational (TaCS-CoE), the Head of the KMUTT Fixed Point Theory and Applications Research Group (KMUTT-FPTARG), and the Director of the Computational and Applied Science for Smart Innovation Cluster (CLASSIC Research Cluster), KMUTT. He is one of the founders of the International Working Group on Applied Nonlinear Analysis (WG-ANA). He has been the Head Leader of the Project CaRe NETWORK: Global Research Network Center under the Global Leader Project by PMU-B, since 2023. He works in computational science and simulation, and computational fluid dynamics (CFD). He has provided and developed many mathematical tools in his field productively over the past years. He has over 1,200 scientific articles and projects either presented or published. He is a Thai Mathematician recognized for his research in the areas of fixed point theory and optimization algorithms with applications in multi-disciplinary areas. He is one of the founders of multi-disciplinary fixed point iteration for computational science, and applied to the image processing area. His research interests include fixed point theory, variational analysis, optimization theory, approximation theory, fractional differential equations, differential games, fuzzy soft set, mathematical modeling for fluid dynamics and inverse problems, dynamic games in economics, traffic network equilibria, bandwidth allocation problem, wireless sensor networks, image restoration, signal and image processing, game theory, and cryptology. Moreover, he is on the editorial board of more than 50 journals and also delivers many invited talks at different international conferences every year, all around the world.



KANOKWAN SITTHITHAKERNGKIET received the B.Sc., M.Sc., and Ph.D. degrees from Naresuan University, Thailand. She is currently an Associate Professor with the Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok (KMUTNB). Her research interests include optimization, nonlinear mappings, least squares methods, optimization problems, image processing, fractional and fractal derivatives, numerical, analytical, and exact solutions.

...